

Rapport Technique

Développement d’un Jeu Stratégique en Java (POO)

Suivant

Sommaire

1. Introduction
2. Présentation générale du jeu
3. Analyse des besoins
4. Conception du système
5. Description des composants principaux
6. Interface utilisateur
7. Choix techniques et outils utilisés
8. Difficultés rencontrées
9. Conclusion

Suivant

1. Introduction

Ce projet a été réalisé dans le cadre du module de Programmation Orientée Objet. L'objectif principal est de concevoir un jeu stratégique en utilisant le langage Java tout en appliquant les notions vues en cours, notamment l'encapsulation, l'héritage et le polymorphisme.

Le jeu développé repose sur la gestion de ressources, la construction de bâtiments et les combats entre unités. Ce projet nous a permis de mieux comprendre comment organiser un programme de taille moyenne en respectant les principes de la programmation orientée objet.

2. Présentation générale du jeu

2.1 Concept du jeu

Le jeu est un jeu de stratégie dans lequel le joueur contrôle une faction sur une carte divisée en plusieurs cases. Chaque case peut représenter un type de terrain différent. Le joueur doit gérer ses ressources afin de construire des bâtiments, créer des unités et étendre son territoire.

L'objectif du jeu est de développer une stratégie efficace pour dominer la carte et battre l'adversaire contrôlé par l'ordinateur.

2.2 Type de jeu

- Jeu de stratégie
- Mode solo (joueur contre machine)
- Jeu fonctionnant au tour par tour ou en temps réel selon l'implémentation

Suivant

3. Analyse des besoins

3.1 Besoins fonctionnels

Le jeu doit permettre :

- La création et l'affichage d'une carte de jeu
- La gestion de différentes ressources (or, bois, pierre, nourriture)
- La construction de bâtiments
- La création et la gestion d'unités
- Le déplacement des unités sur la carte
- La gestion des combats entre unités
- L'affichage des informations du joueur

3.2 Besoins non fonctionnels

- Respect des principes de la programmation orientée objet
- Code organisé et lisible
- Architecture permettant d'ajouter facilement de nouvelles fonctionnalités
- Bon fonctionnement général du jeu

4. Conception du système

4.1 Organisation générale du projet

Le projet est organisé sous le package principal com.stickwar.legacy, qui regroupe l'ensemble des composants du jeu. Cette organisation permet de séparer clairement les responsabilités de chaque partie du système et de faciliter la compréhension du code.

Le package core contient les classes centrales du jeu. La classe GameMain représente le point d'entrée de l'application et gère le lancement du jeu. La classe GameState permet de gérer les différents états du jeu (en cours, victoire ou défaite). La gestion du déroulement du jeu est assurée par la classe TurnManager (ou RealTimeManager selon le mode choisi).



Le package entities regroupe toutes les entités présentes dans le jeu. Une classe abstraite Entity définit les attributs communs comme la position, les points de vie et un identifiant unique. La classe abstraite Unit hérite de Entity et ajoute des caractéristiques spécifiques aux unités comme l'attaque, la vitesse de déplacement et le propriétaire.

Les bâtiments sont regroupés dans le sous-package entities.buildings. Une classe abstraite Building définit la base commune des bâtiments, tandis que des classes concrètes comme Base, GoldMine et Tower implémentent des comportements spécifiques.

Les autres packages permettent de séparer la gestion des ressources, de la carte, de la création des unités, de l'interface utilisateur et des utilitaires.

4.2 Conception orientée objet

La conception du jeu repose fortement sur l'héritage et les classes abstraites. Les classes Entity, Unit et Building permettent de factoriser les attributs et comportements communs.

Le polymorphisme est utilisé pour manipuler les différentes unités (Miner, Swordwrath, Archidonis) de manière générique à travers le type Unit. L'encapsulation est respectée grâce à l'utilisation d'attributs privés et de méthodes d'accès.

Cette approche permet d'ajouter facilement de nouvelles unités ou bâtiments sans modifier la structure existante du projet.

5. Description des composants principaux

5.1 Gestion de la carte et du territoire

La carte du jeu est représentée par la classe GameMap, située dans le package world. Elle est composée de plusieurs secteurs générés de manière procédurale. Chaque case de la carte est représentée par la classe Tile, qui contient des informations telles que son type, son propriétaire et son accessibilité.

La classe TerritoryManager est responsable de la gestion des territoires. Elle calcule la ligne de front et met à jour le propriétaire des cases en fonction de l'avancée des unités.

Suivant

5.2 Gestion des ressources

Les ressources du jeu sont gérées par la classe ResourceManager, située dans le package resources. Les quantités de ressources sont stockées à l'aide d'une structure Map<String, Integer>, permettant de gérer facilement l'or, le bois, la pierre et la nourriture.

Ces ressources sont consommées lors de la création des unités et de la construction des bâtiments.

5.3 Unités

Les unités du jeu héritent toutes de la classe abstraite Unit. Trois types d'unités sont implémentés :

- Miner, utilisé pour la collecte de ressources
- Swordwrath, unité de combat rapproché
- Archidonis, unité de combat à distance

Chaque unité possède des caractéristiques propres, tout en partageant une structure commune.

5.4 Bâtiments

Les bâtiments héritent de la classe abstraite Building. Le jeu comprend notamment :

- Base, qui représente la structure principale du joueur
- GoldMine, permettant la production d'or
- Tower, utilisée pour la défense du territoire

Chaque bâtiment a un rôle précis et interagit avec les ressources du joueur.

5.5 Création des unités

La création des unités est centralisée dans la classe UnitFactory, située dans le package factories. Cette classe permet d'instancier les différentes unités du jeu de manière contrôlée, ce qui simplifie la gestion et l'extension du système.

Suivant

6. Interface utilisateur

L’interface utilisateur comprend un menu principal ainsi qu’un affichage de la carte et des informations du joueur, comme les ressources disponibles et les unités présentes.

Des messages sont également affichés pour informer le joueur des événements importants.

7. Choix techniques et outils utilisés

Le jeu a été développé en Java (version 11 ou supérieure). Le projet utilise une architecture orientée objet claire basée sur des classes abstraites, des packages bien définis et des collections Java.

Les environnements de développement utilisés sont IntelliJ IDEA, Eclipse ou Visual Studio Code. Les collections List, Map, HashMap et ArrayList sont utilisées pour gérer les données du jeu, notamment les ressources et les notifications.

Une classe utilitaire Constants est utilisée pour centraliser les valeurs fixes comme les types de cases et les clés des ressources, ce qui améliore la lisibilité et la maintenabilité du code.

8. Difficultés rencontrées

Au cours de la réalisation de ce projet, certaines difficultés ont été rencontrées, notamment l’organisation du code et la mise en place du système de combat.

Ces difficultés ont été surmontées grâce au travail en groupe et à une meilleure compréhension des concepts de la programmation orientée objet.

Suivant

9. Conclusion

Ce projet nous a permis d’appliquer concrètement les notions de programmation orientée objet vues en cours. Il nous a également aidés à améliorer notre capacité à travailler en groupe et à organiser un projet informatique.

Le jeu réalisé respecte les exigences du cahier des charges et peut être amélioré dans le futur par l’ajout de nouvelles fonctionnalités ou d’une interface graphique plus avancée.